Introduzione al Linguaggio R

Analisi Dati e Statistica, 2026–27



Paolo Bosetti

Ultimo aggiornamento: 29/10/2026

Indice

1 Link alle slide **2**

2 Introduzione a R **3**

2.1 Link utili . . . 3
2.2 Ambiente Positron . . . 3
2.3 Il linguaggio R . . . 3
2.4 Assegnazioni . . . 4
2.5 Tipi, o classi native . . . 4
2.6 Valori speciali . . . 5
2.7 Coercizione . . . 5
2.8 Vettori . . . 6
2.9 Vettori . . . 6
2.10 Facciamo qualche prova . . . 7
2.11 Introspezione . . . 7
2.12 Matrici . . . 7
2.13 Fattori . . . 8
2.14 Stringhe . . . 9
2.15 Indicizzazione . . . 9
2.16 Indicizzazione . . . 10
2.17 Facciamo qualche prova . . . 10
2.18 Funzioni . . . 11
2.19 Funzioni freccia (*replacement functions*) . . . 12
2.20 Controllo di flusso . . . 12
2.21 Esercizio . . . 12
2.22 Argomenti delle funzioni . . . 13

2.23 Differenza tra <- e = . . . 14
2.24 Dataframe . . . 14
2.25 Dataframe . . . 15
2.26 Esercizio . . . 15
2.27 Liste . . . 15
2.28 Algoritmi di uso comune . . . 16
2.29 Ordinamento di vettori . . . 16
2.30 Ordinamento di dataframe . . . 17
2.31 Campionamento . . . 17
2.32 Griglie . . . 18
2.33 Esercizio . . . 18
2.34 Aggregazione . . . 19
2.35 Tabelle di contingenza . . . 19
2.36 Tabelle di contingenza . . . 20
2.37 Input/output su file . . . 21
2.38 Input da file . . . 21
2.39 Input da file . . . 21
2.40 Input da file CSV . . . 22
2.41 Output su file . . . 22
2.42 Tidyverse . . . 23
2.43 Tidyverse . . . 23
2.44 Notazione infissa . . . 23
2.45 Notazione infissa, sequenziata . . . 24
2.46 Notazione prefissa . . . 24
2.47 Notazione prefissa . . . 25
2.48 Help in linea . . . 25

```
options(width = 60)
set.seed(0)
```

1 Link alle slide

Questo contenuto è anche disponibile in formato PDF

2 Introduzione a R

L'analisi statistica richiede l'uso di software specifico

Oggi i due software/linguaggi più utilizzati in questo campo sono Python e R, seguiti da Matlab

Noi utilizzeremo R perché specifico per la statistica, orientato alla grafica e open source

2.1 Link utili

- GNU-R: https://cran.mirror.garr.it/CRAN/
- Positron: https://positron.posit.co
- RStudio: https://posit.co/downloads/
- Cheat sheet: https://posit.co/resources/cheatsheets/
- Tidyverse: https://tidyverse.org
- Materiale corso: https://github.com/pbosetti/ADAS-26

2.2 Ambiente Positron

- Installazione: prima R, poi Positron
- **Solo su windows**, installare anche Rtools
- Positron lavora su progetti, o **cartelle**
- Un progetto contiene anche impostazioni specifiche e comuni ai file nella cartella
- Una **sessione** di Positron può operare su un unico progetto
- Si possono aprire più sessioni contemporaneamente
- Positron è un ambiente molto potente e complesso, adatto anche alla compilazione di *report* tecnici, articoli, libri e presentazioni (come questa), e può gestire anche linguaggi diversi da R (come Python, C, C++, ecc.)

2.3 Il linguaggio R

- R è un linguaggio ad alto livello, declarativo, interpretato, a sintassi C-like
- R è sia un linguaggio, sia un interprete
- R è un *dynamically typed language*
- R è utilizzato sia in **modalità script** che in **modalità interattiva**
- R è nato come versione GNU open source di S, un linguaggio proprietario per analisi statistiche
- Positron è una IDE Open Source per R, basata su Visual Studio Code
- Positron è l'alternativa moderna a **RStudio**

2.4 Assegnazioni

Ogni linguaggio usa delle **variabili** per memorizzare valori ed oggetti mediante un'operazione di **assegnazione**:

```
a <- 1
# ma anche
b = 2
# tuttavia si preferisce la notazione a freccia,
# perché funziona anche così:
3 -> c
# per visualizzare il valore di una variabile:
c
```

```
[1] 3
```

```
# in un colpo solo, assegnazione e visualizzazione:
(d <- "stringa")
```

```
[1] "stringa"
```

L'esecuzione di un comando fornisce direttamente un risultato:

```
12*12
```

```
[1] 144
```

Si noti il testo `[1]` all'inizio della riga di output: sarà chiaro più avanti

2.5 Tipi, o classi native

- R ha 6(+1) tipi o *classi* native
  - character: `"a"`, `"string"`, `'my text'`
  - numeric: `1`, `3.1416`
  - integer: `1L`
  - logical: `TRUE`, `FALSE` (oppure `T` e `F`)
  - complex: `1+4i`
  - function: una *funzione*
  - (raw: sequenza di bit)
- Ogni istanza è intrinsecamente un vettore
- Uno scalare è semplicemente un vettore di lunghezza 1

2.6 Valori speciali

- Sono definiti i seguenti valori speciali:
  - `NA`: valore ignoto (Not Available)
  - `NULL`: niente
  - `Inf`: Infinito
  - `NaN`: Not a Number (esempio `0/0`)

```
length(c(1, NA, 3))   # c(...) è un vettore
```

```
[1] 3
```

```
length(c(1, NULL, 3)) # NULL è come se non ci fosse!
```

```
[1] 2
```

2.7 Coercizione

- Quando si mescolano tipi differenti, ad es. in un vettore, R li trasforma in un tipo comune:

```
c(1L, 7, "2")
```

```
[1] "1" "7" "2"
```

```
c(T, 0)
```

```
[1] 1 0
```

```
as.numeric(c("a", "1"))
```

```
Warning: NAs introduced by coercion
```

```
[1] NA  1
```

```r
as.character(c(1, 1.7))
```

```
[1] "1"   "1.7"
```

2.8 Vettori

```r
# Si costruiscono con l'operatore/funzione c():
v1 <- c(10, 2, 7.5, 3)
# oppure con una sequenza:
v2 <- 1:10
# anche con passo specificato:
v3 <- seq(1, 10, 0.5)
# Le funzioni si chiamano con le parentesi tonde,
# separando argomenti con ,
```

```r
v1 <- c(10, 2, 7.5, 3)
v2 <- 1:10
v3 <- seq(1, 10, 0.5)
```

Si noti in questo caso l'output per `v3`:

```r
(v3 <- seq(1, 10, 0.5))
```

```
 [1]  1.0  1.5  2.0  2.5  3.0  3.5  4.0  4.5  5.0  5.5  6.0
[12]  6.5  7.0  7.5  8.0  8.5  9.0  9.5 10.0
```

Il primo elemento della prima riga è l'elemento `[1]` del vettore, mentre il primo elemento della seconda riga è l'elemento `[12]`. In tutto, il vettore `v3` ha 19 elementi

2.9 Vettori

Le variabili sono nativamente dei vettori. Gli scalari sono solo vettori di dimensione 1:

```r
a <- 10
length(a)
```

```
[1] 1
```

```r
length(v3)
```

```
[1] 19
```

Le funzioni e gli operatori agiscono quindi sempre su vettori (sono *vettorializzati*):

```r
a * 2
```

```
[1] 20
```

```r
v3 + 2
```

```
 [1]  3.0  3.5  4.0  4.5  5.0  5.5  6.0  6.5  7.0  7.5  8.0
[12]  8.5  9.0  9.5 10.0 10.5 11.0 11.5 12.0
```

2.10 Facciamo qualche prova

```r
(v1 <- c(7, 2.5, 3, 1:10))
v1[c(3,7,2)]
```

2.11 Introspezione

- Funzioni utili per ispezionare gli oggetti:
  - `mode()`: *storage mode*
  - `class()`: classe (alto livello, uguale a `mode()` per tipi base)
  - `typeof()`: tipo (basso livello)
  - `length()`: lunghezza vettore
  - `attributes()`: metadati
  - `str()`: struttura di un oggetto
  - `summary()`: riassunto statistico

2.12 Matrici

- Si costruiscono con la funzione `matrix()`

```r
(m1 <- matrix(1:10, 2, 5))
```

- la funzione `array()` costruisce matrici n-dimensionali
- Una matrice è un vettore con attributo `dim`:

```r
attr(m1, "dim")
v <- 1:4
attr(v, "dim") <- c(2,2) # equivale a dim(m) <- c(2,2)
v
```

```r
m1 <- matrix(1:10, 2, 5)
```

2.13 Fattori

- Una classe aggiuntiva (non base) ma molto comune è `factor`
- Rappresenta variabili categoriche (ordinate o non)

```r
(vf <- factor(LETTERS[1:5], levels=LETTERS[c(2, 1, 3, 5, 4)], ordered=T))
```

```
[1] A B C D E
Levels: B < A < C < E < D
```

```r
class(vf)
```

```
[1] "ordered" "factor"
```

```r
typeof(vf)
```

```
[1] "integer"
```

```r
vf[1] < vf[3]
```

```
[1] TRUE
```

2.14 Stringhe

Una stringa può essere pensata come un vettore di caratteri di lunghezza maggiore di 1.

Le funzioni di manipolazione di stringhe più comuni sono `cat()`, `paste()` e `paste0()`. La prima serve a stampare la stringa tale e quale:

```
cat("Ciao!")
```

```
Ciao!
```

Le due funzioni `paste()` e `paste0()` servono a unire due o più stringhe, la prima inserendo uno spazio in mezzo, la seconda senza spazio:

```
paste("Ciao,", "Mondo!")
```

```
[1] "Ciao, Mondo!"
```

```
paste0("Ciao,", "Mondo!")
```

```
[1] "Ciao,Mondo!"
```

2.15 Indicizzazione

- La sintassi di indicizzazione di R è molto flessibile e potente
- si usano sempre le parentesi quadre `[r,c]`, la **base è 1**
- se un indice manca, significa "tutte le righe|colonne"

```
v3[3]
```

```
[1] 2
```

```
m1[1,1]
```

```
[1] 1
```

```r
m1[2,]
```

```
[1]  2  4  6  8 10
```

```r
m1[,]
```

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    3    5    7    9
[2,]    2    4    6    8   10
```

2.16 Indicizzazione

- Un indice può essere anche un vettore di posizioni o un vettore di valori booleani

```r
v1[c(2,4,1)] # estrae solo gli elementi 2, 4, e 1
```

```
[1]  2  3 10
```

```r
v2[v2 %% 2 == 0] # estrae gli elementi divisibili per 2
```

```
[1]  2  4  6  8 10
```

Il secondo caso funziona grazie all'**operatore modulo**:

```r
v2 %% 2 == 0 # operatore modulo (resto)
```

```
 [1] FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE FALSE  TRUE
[10]  TRUE
```

> **Tip**
>
> NOTA: `TRUE` e `FALSE` possono essere abbreviati in `T` e `F`

2.17 Facciamo qualche prova

```r
v1 <- c(10, 2, 7.5, 3)
v2 <- 1:10
v1[c(2,4,1)]
v2[v2 %% 2 == 0]

m1 <- matrix(1:4, nrow = 2, byrow=T)
m2 <- matrix(c(2, 3, 1, 7, 5, 1), nrow = 2, byrow=T)
m1 * m2 # Errore!
```

> **Tip**
>
> Cercare nell'help in linea funzioni come **%*%**

2.18 Funzioni

- Le funzioni sono *first class objects*, cioè sono variabili come altre
- possono essere assegnate a variabili e passate a funzioni

```r
my_fun <- function(x) x^2
my_fun(1:5)
```

```
[1]  1  4  9 16 25
```

```r
your_fun <- my_fun
your_fun(6)
```

```
[1] 36
```

```r
my_apply <- function(x, f) f(x)
my_apply(10, my_fun)
```

```
[1] 100
```

- Se la definizione richiede più righe, si usa un **blocco** tra `{}`
- Ogni funzione ritorna **sempre** l'ultima espressione valutata
- Oppure esplicitamente mediante `return()`

2.19 Funzioni freccia (*replacement functions*)

- Abbiamo visto cose come `dim(v) <- c(2,3)`: come si dichiarano?

```
`pwr<-` <- function(obj, value) obj ** value
a <- 2
pwr(a) <- 10
a
```

```
[1] 1024
```

- L'ultimo argomento **deve** chiamarsi `value` e rappresenta il lato destro dell'assegnazione!

2.20 Controllo di flusso

R supporta le tipiche istruzioni di controllo di flusso

- per istruzioni condizionali:
  - `if(cond) expr`
  - `if(cond) true.expr  else  false.expr`
  - `ifelse(cond, true.expr, false.expr)`
- e per i cicli:
  - `for(var in seq) expr`
  - `while(cond) expr`
  - `repeat expr`
  - `break`
  - `next`

2.21 Esercizio

2.21.1 Assegnazione

Scrivere una funzione `sum_odd` che sommi tutti gli elementi con indice dispari di un vettore numerico. La funzione deve avere un argomento `x` e restituire la somma di tutti gli elementi di `x`, usando un ciclo `for`.

```r
sum_odd <-function(x) {
  # scrivi qui il codice
}

sum_odd(1:10)
```

> **Warning**
>
> L'uso del ciclo `for` è sconsigliato in R (perché è più lento di alcune alternative), ma è utile per esercitarsi con il controllo di flusso.

2.21.2 Soluzione

```r
sum_odd <-function(x) {
  odd <- x[c(T, F)]
  sum <- 0
  for (e in odd) {
    sum <- sum + e
  }
  return(sum)
}

sum_odd(1:10)
```

2.22 Argomenti delle funzioni

- Gli argomenti possono essere indicati per posizione o per nome
- Gli argomenti nominati possono comparire in qualsiasi ordine
- Gli argomenti possono avere un default, in tal caso sono opzionali

```r
f <- function(x, y, n=10, test=F) {
  ifelse(test, 0, x^y + n)
}
f(2, 10)
```

```
[1] 1034
```

```r
f(test=F, y=10, x=2)
```

```
[1] 1034
```

```r
f(test=T)
```

```
[1] 0
```

2.23 Differenza tra <- e =

- L'operatore = come assegnazione è valido solo al *top-level*
- L'operatore <- è valido ovunque, anche come argomento di funzione:

```r
system.time(m <- mean(1:1E6))
```

```
   user  system elapsed
  0.001   0.000   0.000
```

```r
m
```

```
[1] 500000.5
```

2.24 Dataframe

- In R più che matrici si usano `dataframe`
- Si tratta di tabelle organizzate per colonne, internamente omogenee ma potenzialmente di tipi differenti

```r
df <- data.frame(A=1:10, B=letters[1:10])
head(df)
```

```
  A B
1 1 a
2 2 b
3 3 c
4 4 d
5 5 e
6 6 f
```

2.25 Dataframe

- Un dataframe può essere indicizzato come una matrice (due indici)
- Oppure selezionando una colonna con la notazione `$`

```
df[2,2]
```

```
[1] "b"
```

```
df$B[2]
```

```
[1] "b"
```

Anche in assegnazione:

```
df$C <- LETTERS[1:10]
head(df, 3)
```

```
  A B C
1 1 a A
2 2 b B
3 3 c C
```

2.26 Esercizio

Dal data frame `mtcars`, estrarre le righe corrispondenti ai veicoli con cilindrata (`disp`) maggiore di 200 e peso minore di 3.5 tonnellate (`wt`)

```
head(mtcars)
```

2.27 Liste

Una lista è una sequenza di coppie chiave-valore, cioè una sequenza di valori identificati da un nome, o chiave.

A differenza dei vettori (che sono **sempre** omogenei) possono contenere valori eterogenei.

```r
(l <- list(A="uno", B="due", C=1:4))
```

```
$A
[1] "uno"

$B
[1] "due"

$C
[1] 1 2 3 4
```

Una lista può essere indicizzata in tre modi:

- con l'operatore `$`: si estrae un unico elemento per nome
- con l'operatore `[]`: si estraggono elementi per posizione **e si ottiene una lista**
- con l'operatore `[[]]`: si estrae un unico elemento per posizione

2.28 Algoritmi di uso comune

- Ordinamento: `sort`, `rev`, `order`
- Campionamento: `sample`, `expand.grid`
- Aggregazione: `by`, `aggregate`
- Tabelle di contingenza: `table`

2.29 Ordinamento di vettori

Per ordinare un vettore si usa la funzione `sort`:

```r
v <- runif(5, 1, 10)
sort(v)
```

```
[1] 3.389578 4.349115 6.155680 9.070275 9.173870
```

```r
rev(sort(v))
```

```
[1] 9.173870 9.070275 6.155680 4.349115 3.389578
```

```r
sort(v, decreasing = T)
```

```
[1] 9.173870 9.070275 6.155680 4.349115 3.389578
```

2.30 Ordinamento di dataframe

Per riordinare un data frame si estraggono gli indici ordinati:

```r
df <- data.frame(A=1:5, B=runif(5))
df[order(df$B),]
```

```
  A         B
1 1 0.2016819
5 5 0.6291140
4 4 0.6607978
2 2 0.8983897
3 3 0.9446753
```

La funzione `order` ritorna appunto gli indici di un vettore ordinati secondo i valori:

```r
order(df$B)
```

```
[1] 1 5 4 2 3
```

dove il primo è l'indice del valore più piccolo di `df$B` e l'ultimo l'indice del più grande

2.31 Campionamento

Campionare un insieme di dati (un vettore) significa estrarre un sottoinsieme (detto **campione**) di valori in maniera casuale. Si esegue con la funzione `sample`:

```r
sample(1:10) # senza reinserimento
```

```
 [1]  2  3  1  5  7 10  6  4  9  8
```

```r
sample(1:10, replace = T) # con reinserimento
```

```
 [1]  9  5  5  9  9  5  5  2 10  9
```

La dimensione del campione può essere uguale (caso sopra) o più piccola dell'insieme iniziale:

```
sample(1:10, size = 5)
```

```
[1] 1 4 3 6 2
```

```
sample(10) # generazione interi casuali senza ripetizione
```

```
 [1] 10  6  7  4  8  9  2  1  3  5
```

2.32 Griglie

Una **griglia** è una matrice che contiene tutte le combinazioni (ordinate) tra n vettori di dimensioni possibilmente diverse. In R viene rappresentata come un data frame e costruita con la funzione `expand.grid`:

```
(df <- expand.grid(A=1:2, B=c("-", "+"), D=c("a", "b", "c")))
```

```
   A B D
1  1 - a
2  2 - a
3  1 + a
4  2 + a
5  1 - b
6  2 - b
7  1 + b
8  2 + b
9  1 - c
10 2 - c
11 1 + c
12 2 + c
```

2.33 Esercizio

Riordinare il dataframe `df` in maniera casuale:

```r
df <- expand.grid(A=1:3, B=LETTERS[1:2])
# aggiungo una colonna di numeri casuali
# riordino df per quella colonna
df # stampo df
```

2.34 Aggregazione

Per aggregazione si intende raggruppare righe aventi elementi comuni in un data frame e applicare ad ogni gruppo una data funzione. È utile ad esempio per il calcolo di sub-totali.

In R può essere eseguita mediante la funzione `by` o la funzione `aggregate` (cambia il tipo di output):

```r
by(df$A, INDICES = df$B, FUN=sum)
```

```
df$B: -
[1] 9
------------------------------------------------
df$B: +
[1] 9
```

```r
aggregate(A~B, data = df, FUN = sum)
```

```
  B A
1 - 9
2 + 9
```

2.35 Tabelle di contingenza

Una tabella di contingenza conta le occorrenze tra una coppia di colonne in un data frame:

```r
head(airquality, n = 3)
```

```
  Ozone Solar.R Wind Temp Month Day
1    41     190  7.4   67     5   1
2    36     118  8.0   72     5   2
3    12     149 12.6   74     5   3
```

```r
with(airquality, table(OzHi = Ozone > 80, Month,
                       useNA = "ifany"))
```

```
       Month
OzHi     5  6  7  8  9
  FALSE 25  9 20 19 27
  TRUE   1  0  6  7  2
  <NA>   5 21  5  5  1
```

> Tip
>
> **NOTA**: `with()` serve per risparmiarsi di scrivere `airquality$Ozone` e `airquality$Month`: rende *implicito* il nome del data frame

2.36 Tabelle di contingenza

- È anche utile `tapply()`, che opera su una tabella analogamente alle funzioni di aggregazione:

```r
round(with(airquality,
           tapply(Ozone, Month, mean, na.rm=T)), 1)
```

```
   5    6    7    8    9
23.6 29.4 59.1 60.0 31.4
```

Con `aggregate()` si farebbe:

```r
aggregate(Ozone~Month, data=airquality, FUN=mean, ra.rm=T)
```

```
  Month    Ozone
1     5 23.61538
2     6 29.44444
3     7 59.11538
4     8 59.96154
5     9 31.44828
```

2.37 Input/output su file

Dato che la statistica si occupa generalmente di grandi quantità di dati è fondamentale poter importare ed esportare dati in formati generici.

Generalmente i dati sono presentati in forma tabulare (per righe e colonne)

I formati più semplici e comuni sono:

- *Flat File*: un file di testo ASCII contenente valori in riga e colonna; le colonne possono essere separate
  - a lunghezza fissa
  - mediante caratteri separatori
- CSV (*Comma-Separated Values*): una versione speciale di FF in cui i campi colonna sono separati da virgole

2.38 Input da file

Un Flat File con campi separati da spazi può avere questo aspetto:

```
# Dati raccolti il 10/8/2023
x y z
1.2 3.7 2.7
2.1 2.5 3.9
3.8 2.2 6.8
```

Un simile file può essere importato come data frame in questo modo:

```
df <- read.table("data_file.txt", header=T, sep=" ", comment.char="#")
```

La funzione `read.table()` dispone di numerose opzioni che consentono di gestire tutti i possibili casi in cui file contenga campi separati da caratteri specifici (spazi od altro)

2.39 Input da file

Un Flat File con campi a larghezza fissa può avere questo aspetto:

```
# Dati raccolti il 10/8/2023
x     y     z
1.2   3.7   2.7
2.1   2.5   3.9
3.8   2.2   6.8
```

Un simile file può essere importato come data frame in questo modo:

```r
df <- read.fwf("data_file.txt", widths=5, header=T, skip=1)
```

Il parametro `skip=1` richiede di saltare la prima linea (commento)

2.40 Input da file CSV

I file CSV sono FF speciali in cui il separatore di campo è la virgola. In questi casi si usa la funzione `read.csv()` che opera analogamente a `read.table()` ma non richiede di specificare il separatore.

Un CSV ha questo aspetto:

```
# Dati raccolti il 10/8/2023
x,y,z
1.2,3.7,2.7
2.1,2.5,3.9
3.8,2.2,6.8
```

Software che usano lingue latine (Italiano, Spagnolo, Portoghese e Francese) adottano la virgola come separatore decimale. Di conseguenza quando questi software (ad. es. MS Excel) generano dei CSV usano il punto e virgola come separatore di campo.

In questo caso da R è necessario utilizzare la funzione `read.csv2()`, che assume la virgola come separatore decimale e il punto e virgola come separatore di campo.

2.41 Output su file

L'operazione opposta all'importazione di un file in un data frame è l'esportazione di un data frame su un file.

Questa operazione viene eseguita con le funzioni opposte alle precedenti:

- `write.table()`
- `write.fwf()`

- `write.csv()` e `write.csv2()`

Tutte queste funzioni hanno due argomenti obbligatori: il data frame da salvare e il file di destinazione:

```
write.csv(df, "data.csv")
```

Altri argomenti opzionali servono per personalizzare il risultato.

2.42 Tidyverse

Assieme a Positron (e RStudio) è emersa una *new wave* di librerie R che modificano radicalmente l'approccio. Vanno sotto il nome collettivo di `tidyverse`

- `ggplot2`: grafici
- `purrr`: programmazione funzionale
- `dplyr`: manipolazione dati
- `stringr`: manipolazione stringhe

- `tibble`: data frame migliorati
- `readr`: importazione dati
- `tidyr`: preparazione dati
- `lubridate`: manipolazione date

2.43 Tidyverse

L'approccio `tidyverse` ha alcune caratteristiche comuni:

- dati in formato **tidy** (un'osservazione per riga; una variabile, o **osservando**, per colonna)
- composizione di funzioni grafiche con `+` (`ggplot(...) + geom_line()`), ogni funzione è un **layer**
- notazione prefissa con `%>%` (`a %>% str()` invece di `str(a)`)

> Tip
>
> È utile consultare i cheat sheet: https://posit.co/resources/cheatsheets/

2.44 Notazione infissa

```r
# creo l'istogramma di un campione di 10 elementi da 100 numeri casuali
# infissa:
hist(sample(rnorm(100), 10))
```

Poco leggibile, il primo passo dell'algoritmo è quello più interno

2.45 Notazione infissa, sequenziata

```r
# creo l'istogramma di un campione di 10 elementi da 100 numeri casuali
# infissa:
hist(sample(rnorm(100), 10))

# infissa sequenziata:
s <- rnorm(100)
c <- sample(s, 10)
hist(c)
```

Più leggibile, l'algoritmo è più evidente, ma richiede la creazione di variabili intermedie

2.46 Notazione prefissa

```r
# creo l'istogramma di un campione di 10 elementi da 100 numeri casuali
# infissa:
hist(sample(rnorm(100), 10))

# infissa sequenziata:
s <- rnorm(100)
c <- sample(s, 10)
hist(c)

# prefissa con pipe:
rnorm(100) %>% sample(10) %>% hist()
```

Molto più leggibile, l'algoritmo sequenziale è evidente, non sono necessarie variabili intermedie

2.47 Notazione prefissa

```r
# creo l'istogramma di un campione di 10 elementi da 100 numeri casuali
# infissa:
hist(sample(rnorm(100), 10))

# infissa sequenziata:
s <- rnorm(100)
c <- sample(s, 10)
hist(c)

# prefissa con pipe:
rnorm(100) %>% sample(10) %>% hist()

# anche su più righe:
rnorm(100) %>%
  sample(10) %>%                                                    ①
  hist                                                              ②
```

① le righe successive alla prima vanno **indentate**
② solo quando si usa pipe, se non ci sono argomenti le parentesi sono opzionali

2.48 Help in linea

- L'help in linea per unafunzione può essere ottenuto con il comando `?<nome funzione>`, ad esempio `?summary` (senza parentesi)
- Alcune funzioni però, come `summary()`, sono **funzioni generiche**, e il loro help è scarno
- Le funzioni generiche dirottano la chiamata a una *funzione specifica* definita dalla classe del primo argomento
- Il nome della funzione specifica è del tipo `summary.lm()`, che è la funzione realmente chiamata quando a passo un modello lineare (es. `summary(df.lm)`)
- Alla fine dell'help per la generica (*See also*) c'è l'elenco delle funzioni specifiche note, che vanno consultate per i dettagli di funzionamento desiderati
- In alternativa si può cercare direttamente `?summary.lm`